

Курсовой проект по курсу дискретного анализа: архиватор

Выполнил студент группы 08-303 МАИ *Арусланов Кирилл*.

Условие

Общая Необходимо реализовать два известных метода сжатия данных для сжатия одного файла. Методы сжатия выбираются из следующих групп:

- Арифметическое кодирование, кодирование по Хаффману
- LZ77, LZW, BWT+MTF+RLE

Формат запуска должен быть аналогичен формату запуска программы gzip. Должны поддерживаться следующие ключи: -c, -d, -k, -l, -r, -t, -1, -9. Должно поддерживаться указание символа дефиса в качестве стандартного ввода.

Вариант задания: LZ77 + кодирование по Хаффману

Метод решения

LZ77 работает по всему тексту, в реализации используется suffix array (SA) для оптимизации поиска совпадений. Сначала входные данные преобразуются в строку с терминатором (маппинг символов для упрощения сортировки). Затем строится suffix array — массив индексов суффиксов, отсортированных лексикографически, с помощью алгоритма на основе удвоения ($O(n \log n)$ время). Далее вычисляется longest common prefix (LCP) для соседних суффиксов, чтобы быстро находить длины совпадений. Поиск наилучшего матча для позиции pos: сканируем соседей в SA, выбирая максимальную длину с минимальным offset. Результат — вектор токенов: либо литерал (offset=0, length=0, ch), либо матч (offset, length, optional ch).

Для уровня -9 токены сериализуются в поток `uint32_t`-символов: литералы как `(unsigned char)ch`, матчи как `UINT32_MAX + offset + length + has_char + optional ch`. Этот поток сжимается каноническим Huffman: считаются частоты, строится дерево (priority queue слияний), собираются длины кодов (обход дерева). Канонизация: сортировка по `(len asc, sym asc)`, вычисление `next_code` для присвоения кодов. Запись: таблица `(unique_count + (sym u32 + len u16)*) + total_symbols u64` + битстрим (MSB-first).

Формат архива: заголовок 8 байт ("`LZ77`" + `version=1` + `level=1/9` + `reserved`), затем для -1 — `u64 token_count` + токены (`u32 offset + u32 length + byte has_char + optional char`); для -9 — Huffman-блок.

Декомпрессия: чтение заголовка, для -1 — чтение токенов фиксированно; для -9 — декодирование Huffman (восстановление кодов, побитовое чтение с накоплением `cur_code`). Затем LZ77-декомпрессия: копирование по `offset/length`.

Ключи обрабатываются в `main`: парсинг `argv`, рекурсивный обход директорий для -r, буферизация `stdin` для -l/-t. List: распаковка в память для расчёта `ratio`. Test: проба декомпрессии без записи.

Описание программы

Программа на C++ разделена на модули:

- `lz77.hpp/cpp` — структура `Token`, сжатие с SA/LCP, декомпрессия.
- `huffman.hpp/cpp` — каноническое Huffman-кодирование (`BitWriter/Reader`, `encode/decode_symbols`).
- `utils.hpp/cpp` — сериализация токенов, поддержка `stdin/stdout`.

- `archiver.hpp/cpp` — функции `compress/decompress_stream`, `list/test_archive`.
- `main.cpp` — парсинг аргументов, обработка файлов/директорий (с `-r`).

Дневник отладки

Изначально Huffman был обычным — перешёл на канонический для компактной таблицы. Ошибки в BitReader/Writer (паddинг, порядок битов). Отлаживал на маленьких тестах (twoletter, lorem). Проверка на stdin требовала буферизации для list/test.

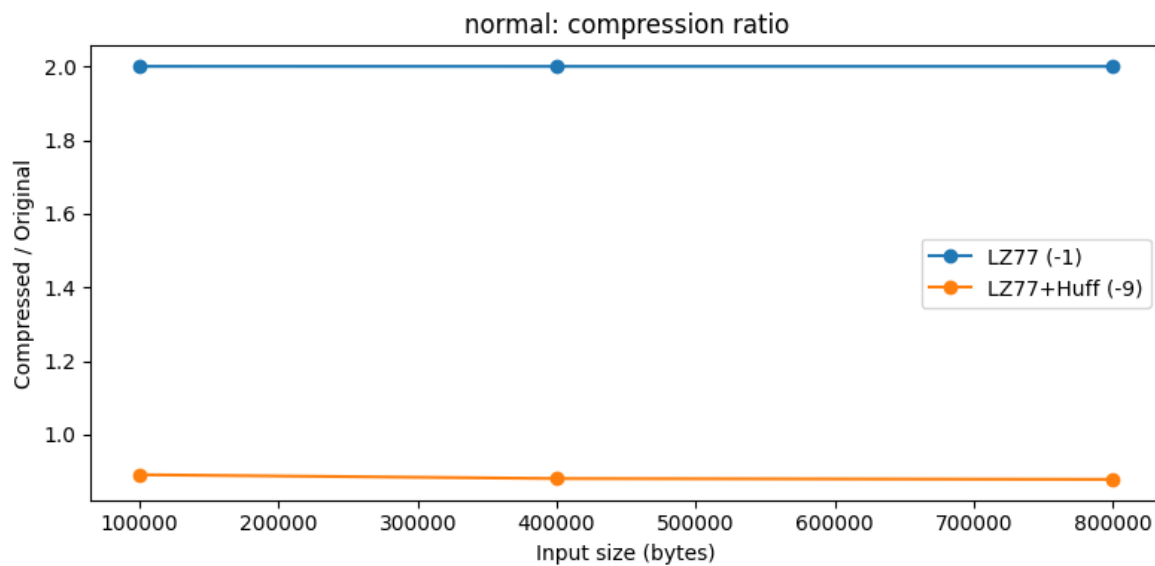
Тест производительности

Тестирование проводилось на синтетических и реальных датасетах. Измерялись время сжатия, время распаковки и коэффициент сжатия (`compressed / original`).

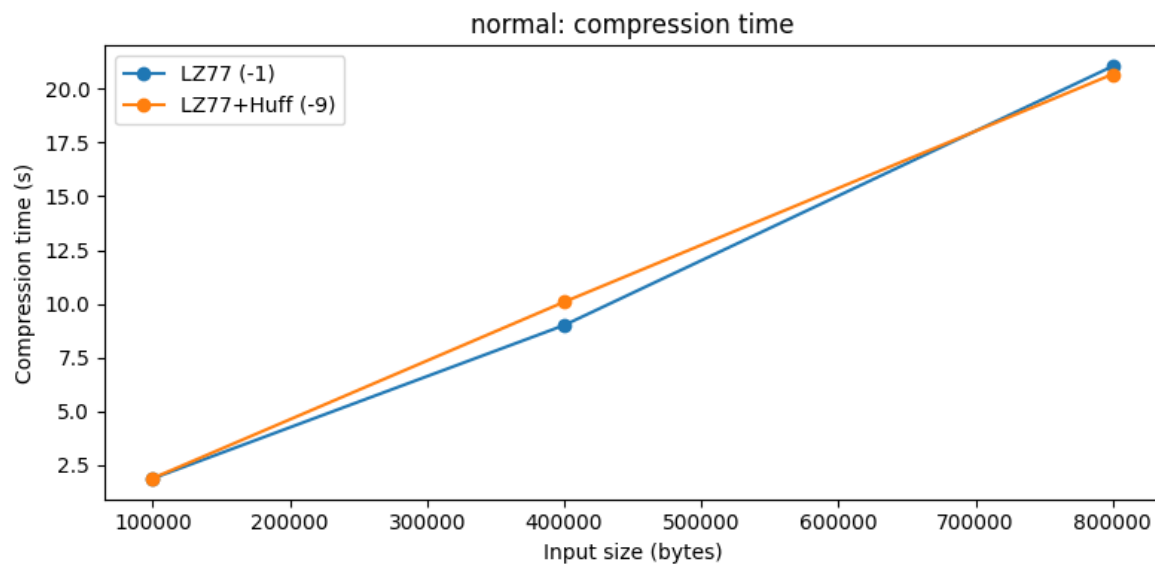
Синтетические данные: нормальное распределение

Результаты для датасета normal

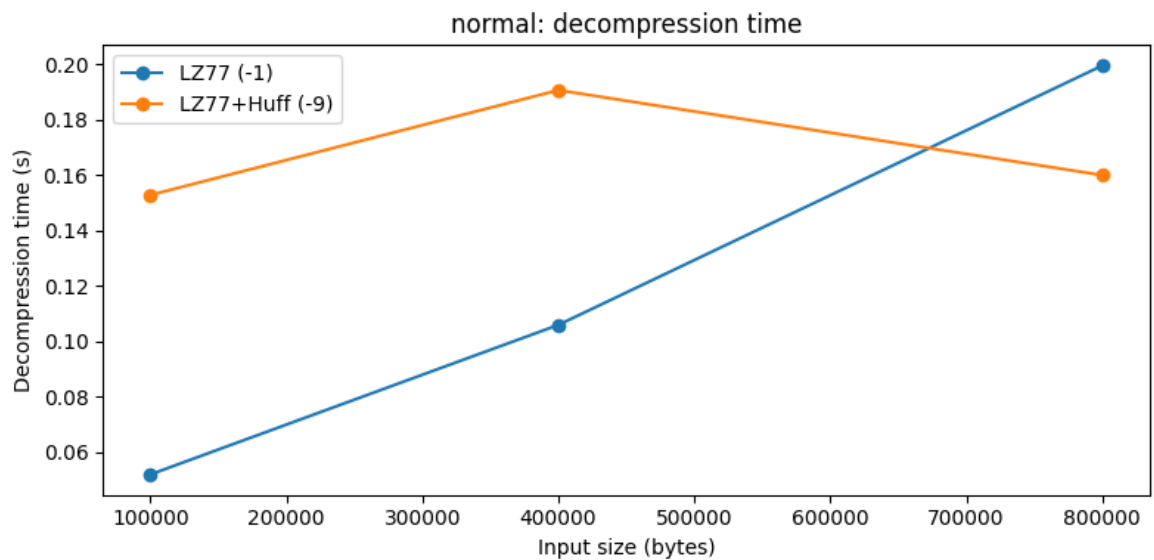
Размер (байт)	Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
100 000	-1	1.876	0.052	200 017
	-9	1.889	0.153	89 136
400 000	-1	9.004	0.106	800 017
	-9	10.086	0.191	352 418
800 000	-1	21.050	0.199	1 600 019
	-9	20.679	0.160	703 034



Коэффициент сжатия (normal)



Время сжатия (normal)



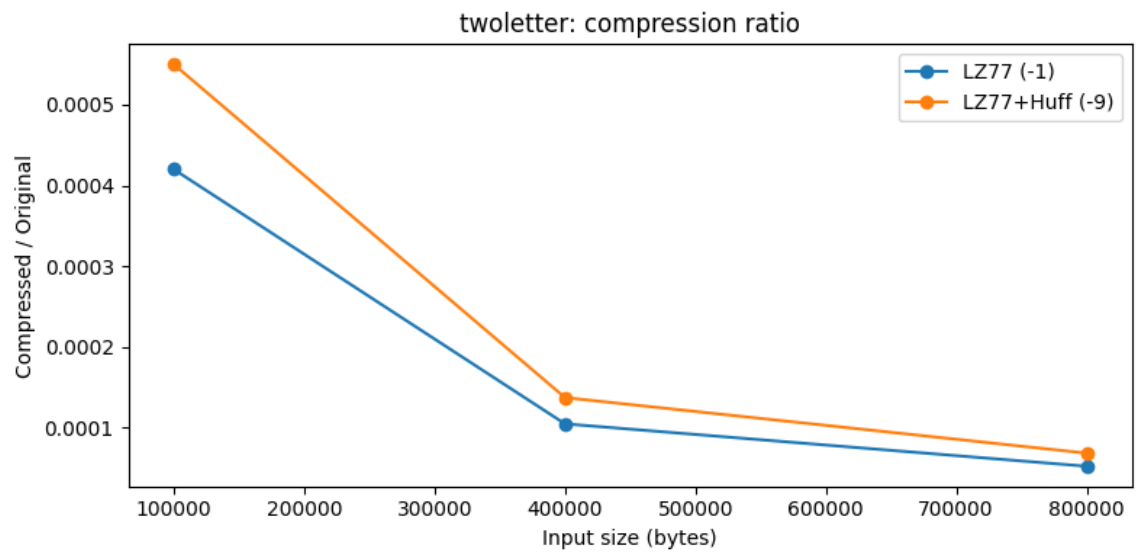
Время распаковки (normal)

Режим -9 показывает отличное сжатие ($\text{ratio} \approx 0.88$ на 800 KB), значительно лучше -1 ($\text{ratio} \approx 2.0$). Это связано с тем, что Huffman эффективно сжимает токены-литералы и малые offset/length. Чистый LZ77 раздувает файл из-за overhead токенов на данных с короткими матчами.

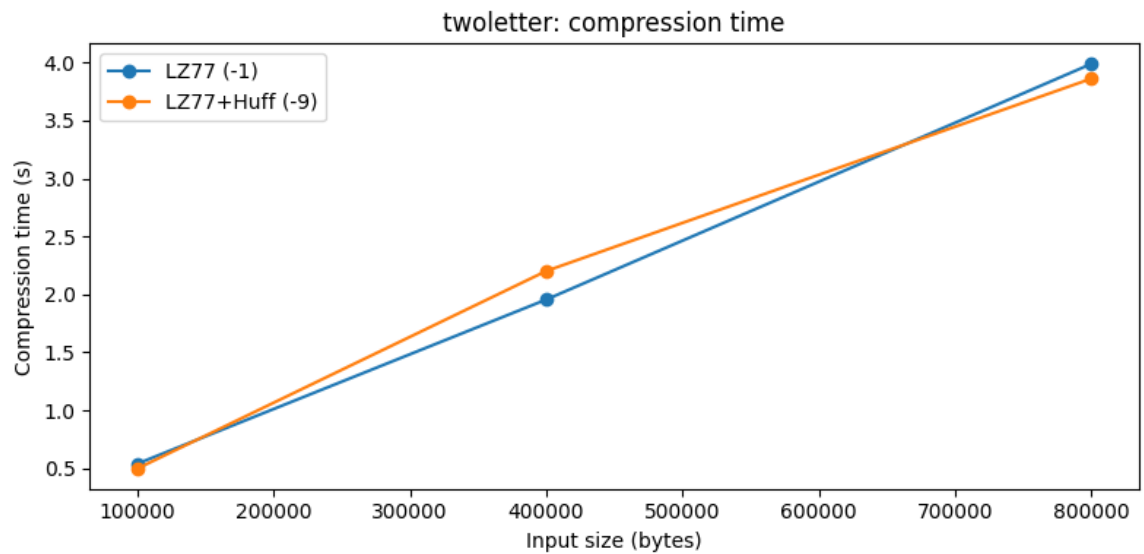
Синтетические данные: высокая повторяемость (twoletter)

Результаты для датасета twoletter

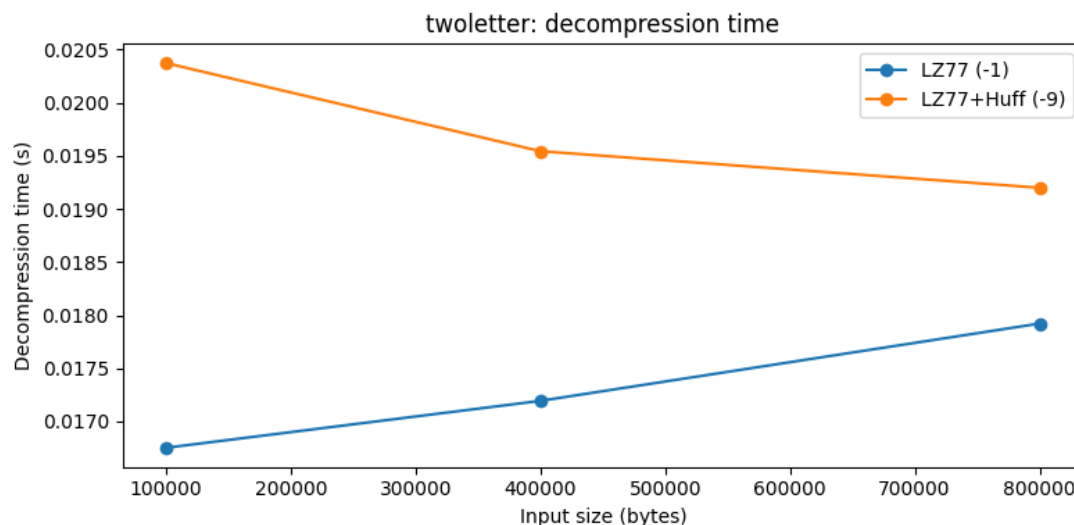
Размер	Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
100 000	-1	0.540	0.017	42
	-9	0.499	0.020	55
400 000	-1	1.956	0.017	42
	-9	2.202	0.020	55
800 000	-1	3.988	0.018	42
	-9	3.861	0.019	55



Коэффициент сжатия (twoletter)



Время сжатия (twoletter)



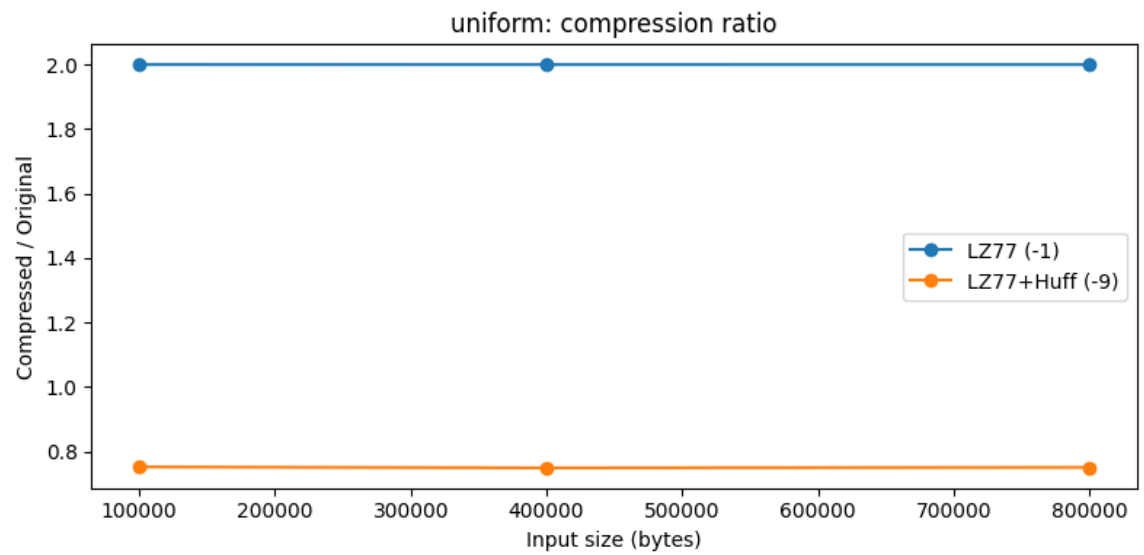
Время распаковки (twoletter)

На данных с высокой повторяемостью оба режима показывают практически идеальное сжатие (размер сжатого файла не зависит от исходного). Режим -1 чуть лучше по размеру, так как не хранит дополнительную информацию для декодирования, но разница минимальна.

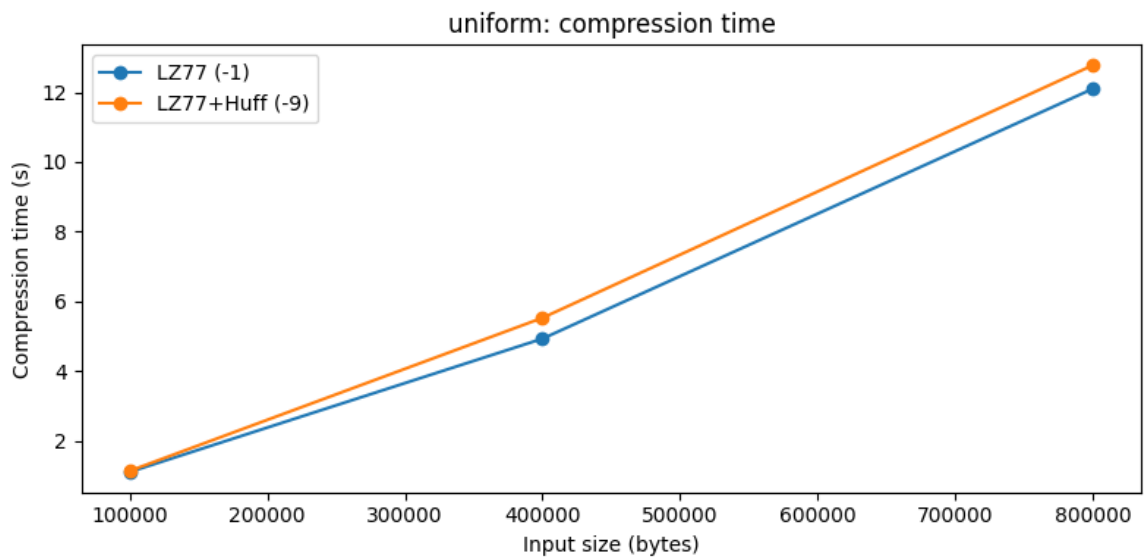
Синтетические данные: равномерное распределение (uniform)

Результаты для датасета *uniform*

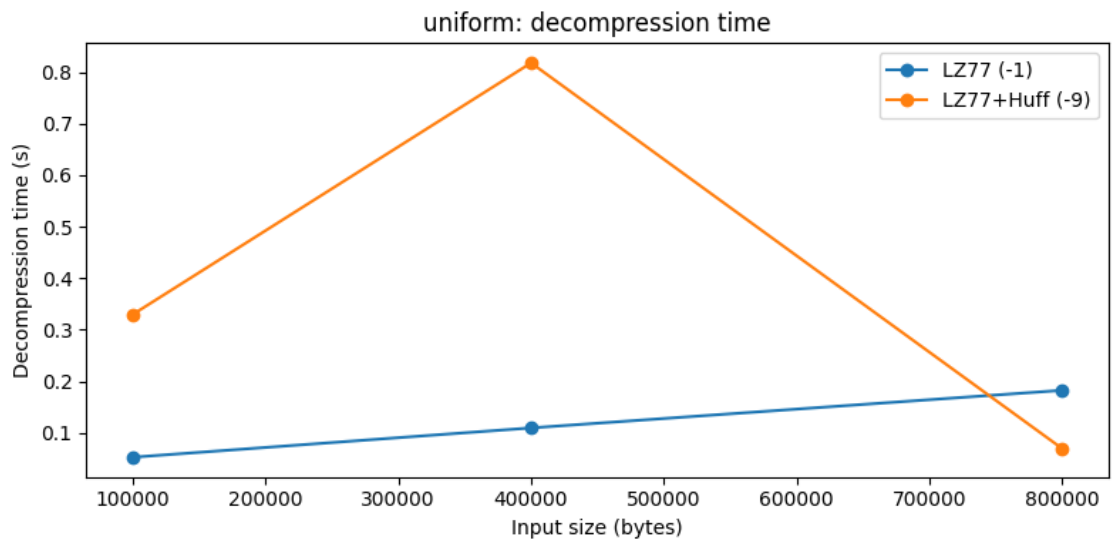
Размер	Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
100 000	-1	1.101	0.053	200 016
	-9	1.134	0.330	75 251
400 000	-1	4.928	0.110	800 016
	-9	5.526	0.818	299 740
800 000	-1	12.101	0.183	1 599 992
	-9	12.776	0.070	600 835



Коэффициент сжатия (uniform)



Время сжатия (uniform)



Время распаковки (uniform)

На равномерных данных -9 даёт хороший результат (ratio ≈ 0.75 на 800 KB), в то время как -1 раздувает файл в 2 раза. Это подтверждает эффективность Huffman на токенах даже при отсутствии длинных матчей.

Реальные данные: текстовая статья (≈ 13 KB)

Результаты для *article.txt*

Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
-1	0.133	0.084	11 236
-9	0.121	0.186	10 092

Режим -9 даёт лучшее сжатие (ratio ≈ 0.76 против 0.85 у -1). На небольшом текстовом файле оба режима эффективны, но Huffman выигрывает за счёт энтропийного кодирования.

Реальные данные: большой текстовый отчёт (≈606 KB)

Результаты для iar_dzz.txt

Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
-1	8.332	0.078	231 489
-9	6.775	0.606	282 574

Здесь неожиданно выигрывает -1 (ratio ≈ 0.38 против 0.47 у -9). Вероятно, причиной этому множество длинных повторяющихся фраз, которые LZ77 покрывает большими матчами, а Huffman добавляет overhead таблицы и менее эффективен на большом количестве уникальных offset/length.

Реальные данные: PNG-изображение (≈1.1 MB)

Результаты для test_image.png

Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
-1	38.297	0.222	2 168 449
-9	35.829	0.215	1 164 273

На уже сжатом PNG -1 раздувает файл почти в 2 раза из-за overhead токенов на короткие матчи. Режим -9 справляется лучше (ratio ≈ 1.04), но всё равно не уменьшает размер.

Таким образом, По степени сжатия архиватор эффективен на данных с повторениями (тексты, отчёты, twoletter). На обычных текстах режим -9 (LZ77 + Huffman) обычно лучше режима -1 (LZ77). На синтетике с нормальным/равномерным распределением -9 даёт ratio ≈ 0.75 – 0.88 против ≈ 2.0 у -1. На уже сжатых данных (PNG) -1 раздувает файл до 2 раз, -9 сохраняет ratio ≈ 1.0 – 1.04 . Комбинация методов хорошо справляется с текстовыми и повторяющимися данными. По скорости сжатие определяется построением suffix array ($O(n \log n)$): на файлах 800 KB — 4–21 секунда. Режим -9 немного

медленнее -1 за счёт Huffman. Распаковка в -1 очень быстрая (0.01–0.2 с даже на мегабайтах), в -9 — в 5–20 раз медленнее из-за побитового ввода-вывода.

Недочёты

- Чтение всего файла в память (read_all) — не подходит для файлов больше RAM.
- Можно добавить эвристики для ускорения.
- Не самая эффективная сериализация для токенов LZ77.

Выводы

Реализованный архиватор сочетает LZ77 и Huffman — классику сжатия без потерь. Отлично работает на текстах с повторениями, приемлемо на случайных и плохо на уже сжатых данных. Основные трудности были связаны с каноническом Huffman и сериализацией.